

学号：202300130183	姓名：宋浩宇
实验名称：实验五：成分句法分析基础实验（CFG / PCFG）	
1. 实验目的	
1. 理解句法分析的基本概念 2. 掌握成分句法树结构 3. 理解上下文无关文法的基本形式 4. 学习使用工具进行句法分析 5. 观察句法歧义现象	
2. 实验原理	
句法分析的目标是分析句子的结构关系。 例如句子：我 爱 自然语言处理 可以表示为：$S \rightarrow NP + VP$ 常见方法：规则方法（CFG）、概率方法（PCFG）、深度学习方法	
3. 实验步骤与代码	
实验步骤： 1. 安装并导入工具 2. 输入句子 3. 生成句法树 4. 可视化结果 5. 分析结构差异	
代码： <pre># -*- coding: utf-8 -*- import stanza from nltk import Tree from collections import Counter, defaultdict data_path = r"./exp-3.1-data.txt" # 读取全部句子 with open(data_path, "r", encoding="utf-8") as f: sentences = [line.strip() for line in f if line.strip()] # 初始化(constituency 依赖 pos) stanza.download("zh-hans", processors="tokenize,pos,constituency", verbose=False) nlp = stanza.Pipeline("zh-hans", processors="tokenize,pos,constituency", use_gpu=False, verbose=False) all_productions = [] all_trees = []</pre>	

```

print("\n===== 全部句子句法树 =====\n")
for idx, sent in enumerate(sentences, 1):
    doc = nlp(sent)
    tree_str = str(doc.sentences[0].constituency)
    tree = Tree.fromstring(tree_str)
    all_trees.append((sent, tree))
    all_productions.extend(tree.productions())
    print(f"[句子 {idx}] {sent}")
    # 关键:按树形结构打印
    tree.pretty_print(unicodelines=True)
    print("-" * 70)

# CFG:全部规则 + 频次
print("\n===== CFG 规则 =====\n")
rule_counter = Counter(str(p) for p in all_productions)
for i, (rule, cnt) in enumerate(sorted(rule_counter.items()), 1):
    print(f"{i:03d}. {rule} [count={cnt}]")

# PCFG:全部规则 + 概率
print("\n===== PCFG 规则 =====\n")
lhs_total = defaultdict(int)
for p in all_productions:
    lhs_total[str(p.lhs())] += 1
pcfg_rows = []
for rule, cnt in rule_counter.items():
    lhs = rule.split(" -> ")[0]
    prob = cnt / lhs_total[lhs]
    pcfg_rows.append((rule, cnt, prob))
for i, (rule, cnt, prob) in enumerate(sorted(pcfg_rows, key=lambda x: (x[0])), 1):
    print(f"{i:03d}. {rule} [count={cnt}, prob={prob:.6f}]")

# 结构组成与差异(每句)
print("\n===== 句子结构组成差异 =====\n")
for idx, (sent, tree) in enumerate(all_trees, 1):
    labels = [t.label() for t in tree.subtrees()]
    c = Counter(labels)
    print(f"[句子 {idx}] {sent}")
    print(
        f"IP={c.get('IP',0)}, NP={c.get('NP',0)},
VP={c.get('VP',0)}, "
        f"CP={c.get('CP',0)}, PP={c.get('PP',0)},
ADJP={c.get('ADJP',0)}, ADVP={c.get('ADVP',0)}"
    )
    print("-" * 70)

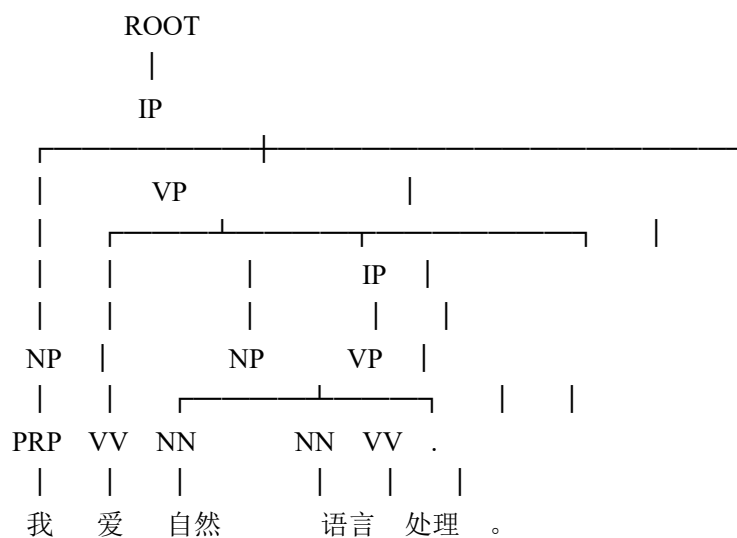
```

4. 实验结果（图表与输出）

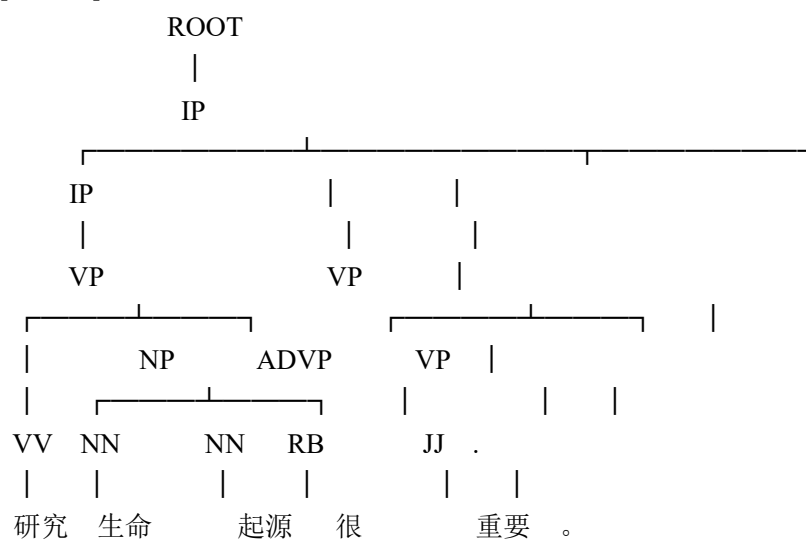
输出结果为：

===== 全部句子句法树 =====

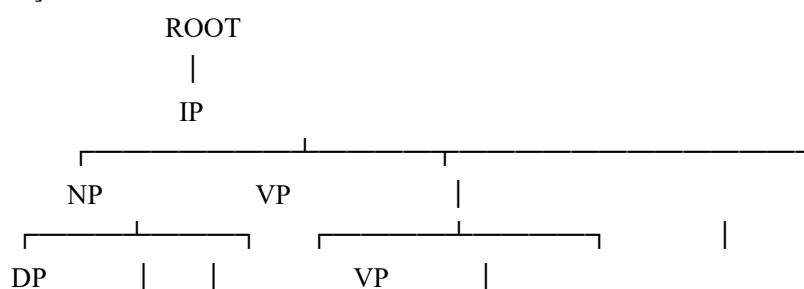
[句子 1] 我爱自然语言处理。

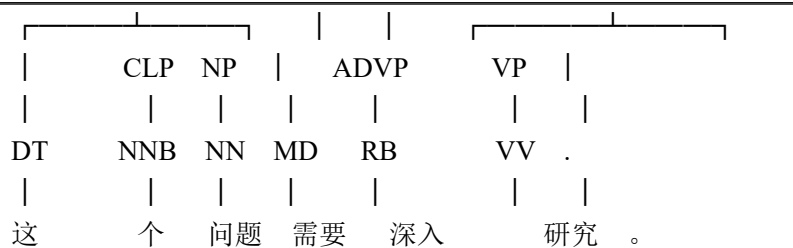


[句子 2] 研究生命起源很重要。

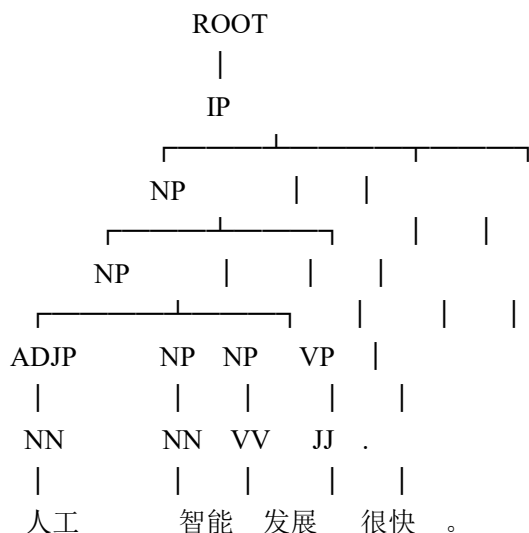


[句子 3] 这个问题需要深入研究。





[句子 4] 人工智能发展很快。



===== CFG 规则 =====

- 001. . -> '。' [count=4]
- 002. ADJP -> NN [count=1]
- 003. ADVP -> RB [count=2]
- 004. CLP -> NNB [count=1]
- 005. DP -> DT CLP [count=1]
- 006. DT -> '这' [count=1]
- 007. IP -> IP VP . [count=1]
- 008. IP -> NP VP . [count=3]
- 009. IP -> VP [count=2]
- 010. JJ -> '很快' [count=1]
- 011. JJ -> '重要' [count=1]
- 012. MD -> '需要' [count=1]
- 013. NN -> '人工' [count=1]
- 014. NN -> '智能' [count=1]
- 015. NN -> '生命' [count=1]
- 016. NN -> '自然' [count=1]
- 017. NN -> '语言' [count=1]

018. NN -> '起源' [count=1]
 019. NN -> '问题' [count=1]
 020. NNB -> '个' [count=1]
 021. NP -> ADJP NP [count=1]
 022. NP -> DP NP [count=1]
 023. NP -> NN [count=2]
 024. NP -> NN NN [count=2]
 025. NP -> NP NP [count=1]
 026. NP -> PRP [count=1]
 027. NP -> VV [count=1]
 028. PRP -> '我' [count=1]
 029. RB -> '很' [count=1]
 030. RB -> '深入' [count=1]
 031. ROOT -> IP [count=4]
 032. VP -> ADVP VP [count=2]
 033. VP -> JJ [count=2]
 034. VP -> MD VP [count=1]
 035. VP -> VV [count=2]
 036. VP -> VV NP [count=1]
 037. VP -> VV NP IP [count=1]
 038. VV -> '发展' [count=1]
 039. VV -> '处理' [count=1]
 040. VV -> '爱' [count=1]
 041. VV -> '研究' [count=2]

===== PCFG 规则 =====

001. . -> '。' [count=4, prob=1.000000]
 002. ADJP -> NN [count=1, prob=1.000000]
 003. ADVP -> RB [count=2, prob=1.000000]
 004. CLP -> NNB [count=1, prob=1.000000]
 005. DP -> DT CLP [count=1, prob=1.000000]
 006. DT -> '这' [count=1, prob=1.000000]
 007. IP -> IP VP. [count=1, prob=0.166667]
 008. IP -> NP VP. [count=3, prob=0.500000]
 009. IP -> VP [count=2, prob=0.333333]
 010. JJ -> '很快' [count=1, prob=0.500000]
 011. JJ -> '重要' [count=1, prob=0.500000]
 012. MD -> '需要' [count=1, prob=1.000000]
 013. NN -> '人工' [count=1, prob=0.142857]
 014. NN -> '智能' [count=1, prob=0.142857]
 015. NN -> '生命' [count=1, prob=0.142857]
 016. NN -> '自然' [count=1, prob=0.142857]
 017. NN -> '语言' [count=1, prob=0.142857]

018. NN -> '起源' [count=1, prob=0.142857]
 019. NN -> '问题' [count=1, prob=0.142857]
 020. NNB -> '个' [count=1, prob=1.000000]
 021. NP -> ADJP NP [count=1, prob=0.111111]
 022. NP -> DP NP [count=1, prob=0.111111]
 023. NP -> NN [count=2, prob=0.222222]
 024. NP -> NN NN [count=2, prob=0.222222]
 025. NP -> NP NP [count=1, prob=0.111111]
 026. NP -> PRP [count=1, prob=0.111111]
 027. NP -> VV [count=1, prob=0.111111]
 028. PRP -> '我' [count=1, prob=1.000000]
 029. RB -> '很' [count=1, prob=0.500000]
 030. RB -> '深入' [count=1, prob=0.500000]
 031. ROOT -> IP [count=4, prob=1.000000]
 032. VP -> ADVP VP [count=2, prob=0.222222]
 033. VP -> JJ [count=2, prob=0.222222]
 034. VP -> MD VP [count=1, prob=0.111111]
 035. VP -> VV [count=2, prob=0.222222]
 036. VP -> VV NP [count=1, prob=0.111111]
 037. VP -> VV NP IP [count=1, prob=0.111111]
 038. VV -> '发展' [count=1, prob=0.200000]
 039. VV -> '处理' [count=1, prob=0.200000]
 040. VV -> '爱' [count=1, prob=0.200000]
 041. VV -> '研究' [count=2, prob=0.400000]

===== 句子结构组成差异 =====

[句子 1] 我爱自然语言处理。

IP=2, NP=2, VP=2, CP=0, PP=0, ADJP=0, ADVP=0

[句子 2] 研究生命起源很重要。

IP=2, NP=1, VP=3, CP=0, PP=0, ADJP=0, ADVP=1

[句子 3] 这个问题需要深入研究。

IP=1, NP=2, VP=3, CP=0, PP=0, ADJP=0, ADVP=1

[句子 4] 人工智能发展很快。

IP=1, NP=4, VP=1, CP=0, PP=0, ADJP=1, ADVP=0

图表结果为:

无图标结果

5. 结果分析

1. 什么是成分句法分析？它的核心思想是什么？

成分句法分析是对句子的整体结构进行分析的方法，它关注词语如何逐层组合成短语，再由

短语组成完整句子。它的核心思想是把句子看成有层级关系的结构，而不是只看词语本身，从而揭示主语、谓语、宾语以及修饰关系在句子中的组织方式。

2. 句法树中的 NP、VP 分别表示什么？

NP 是名词短语，通常用来表示人、事物、概念等名词性成分，比如这个问题和人工智能。VP 是动词短语，通常表示动作、行为或状态，比如需要深入研究和进展很快。在句法树中，NP 和 VP 往往是构成句子主干的关键成分。

3. 同一个句子是否可能有多种句法结构？请举例说明

同一个句子可能有多种句法结构，这就是句法歧义。比如咬死了猎人的狗可以理解为狗咬死了猎人，也可以理解为咬死了猎人的那条狗。两种理解对应不同的短语划分和语法关系，所以会形成不同的句法树。

4. 成分句法分析与词性标注有什么关系？

成分句法分析和词性标注是层层递进的关系。词性标注先给每个词赋予语法类别，比如名词、动词、形容词，然后句法分析再利用这些词性信息去构建更高层的短语和句子结构。可以理解为词性标注是基础，句法分析是在这个基础上做结构建模。

5. 为什么句法分析比词性标注更复杂？

句法分析比词性标注更复杂，因为词性标注通常是对单个词做局部判断，而句法分析需要同时考虑整句的全局结构和多层级组合关系。句法分析还要处理长距离依赖、结构歧义和嵌套短语等问题，因此搜索空间更大，模型设计和计算都更困难。

6. 实验总结

本次实验在方法上并不复杂，真正困难主要集中在环境配置阶段，出现了依赖版本冲突模型下载异常拦截和组件不兼容等问题，导致原方案难以稳定复现。为了保证实验按时完成且结果可运行，最终采用了替代工具链来完成句法树生成可视化结构对比和歧义分析，实验目标仍然完整达成。

LLM 使用情况：

使用 GPT-5.4 帮我解析这恶心人的报错和给我提出了另一个替代方案。